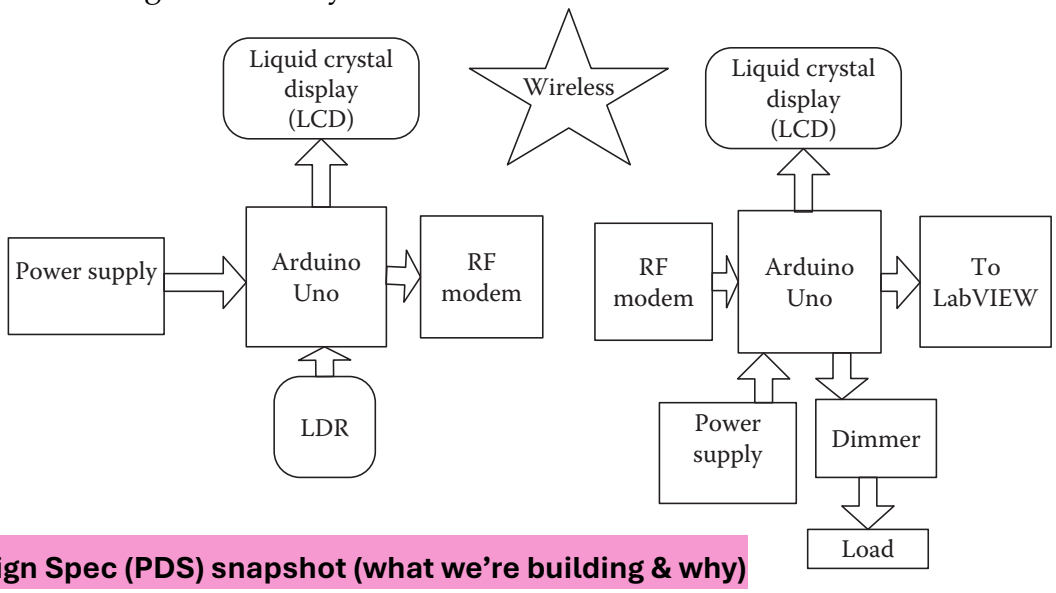
ECE 3709 Lab 7
Fall 2025
Light-Dependent Resistor-Based Light Intensity Control System

Introduction

This project is about to design the light-dependent resistor-based room light intensity control system. The system is designed in two sections: (1) transmitter section and (2) receiver section. At the transmitter, light-dependent resistor (LDR) is connected to the Arduino to sense the intensity of the surroundings. The level of intensity is sensed and transmitted to the receiver section to control the intensity at the required level. The receiver section comprises the Arduino Uno, LCD, RF modem, dimmer, and load (bulb). The load can be controlled in 4, 8, or 16 levels. At the server, a LabVIEW GUI is created to observe the system. Figure below shows the block diagram of the system.



1) Product Design Spec (PDS) snapshot (what we're building & why)

Goal. Sense ambient light with an LDR at a **transmitter** Arduino Uno, send a compact intensity value wirelessly via a 2.4 GHz RF modem, and **receiver** Arduino adjusts a lamp/dimmer in discrete steps to keep room brightness within target levels. Display the live level on 16x02 LCDs; and view data on a LabVIEW GUI.

Key PDS requirements:

- Wireless telemetry: simple, low-latency one-byte level over ~9600 bps serial via 2.4 GHz RF pair.
- Human feedback: 20×4 character LCD at Tx and Rx to show LDR level.
- Actuation: receiver drives a **dimmer** with 4 control lines (expandable to 8/16 steps).
- Power
- PC monitoring: LabVIEW front panel via serial port, 9600-8-N-1.

Why these choices (rationale):

- **Byte-level payload** minimizes RF link complexity and matches the sample code structure in the document.
- **Discrete dimmer steps** (4/8/16) simplify safety & repeatability for students and mirror the book's design intent.
- **LCD feedback** at both ends makes bring-up and debugging observable without a PC.

To design the system, following components are required

TABLE 1 Components List for a Transmitter Section

Component/Specification	Quantity
Arduino Uno (5 V)	1
nRF24L01+ + adapter/base (recommended) or bare module + 3.3 V LDO + level shifters	1
LDR sensor (+ simple divider board/patch as before)	1
I²C 1602 LCD	1
Breadboard/zero PCB, jumpers	1
Single 5 V supply	1

TABLE 2 Components List for a Receiver Section

Component/Specification	Quantity
Arduino Uno (5 V)	1
nRF24L01+ + adapter/base (or 3.3 V LDO + level shifters)	1
I²C 1602 LCD	1
Dimmer driver + load (keep isolation & TA-only mains handling)	1
Breadboard/zero PCB, jumpers	1
5 V supply	1
PC with LabVIEW for serial view @ 9600	1

2) Wiring

2.1 Shared notes

I²C LCD (PCF8574):

VCC → 5 V, GND → GND, SDA → A4, SCL → A5

Typical addresses: 0x27 or 0x3F (we'll show how to change in code).

nRF24L01+ (SPI) on Uno:

SCK → D13, MISO → D12, MOSI → D11 (hardware SPI)

CE and CSN are any digital pins we choose (picked to avoid dimmer pins).

Decoupling: put 0.1 µF + 10–47 µF near the radio's VCC/GND.

2.2 Transmitter (LDR → nRF24 → I²C LCD)

LDR: as in original—output (divider) → A0, +5 V, GND.

LCD (I²C): SDA→A4, SCL→A5, VCC→5 V, GND→GND.

nRF24: VCC→3.3 V (from adapter or LDO), GND→GND, CE→D7, CSN→D8, SCK→D13, MOSI→D11, MISO→D12.

2.3 Receiver (nRF24 → I²C LCD + Dimmer)

LCD (I²C): SDA→A4, SCL→A5, VCC→5 V, GND→GND.

nRF24: VCC→3.3 V, GND→GND, CE→D2, CSN→D3, SCK→D13, MOSI→D11, MISO→D12.

Dimmer control (keep original 4-line idea): D7, D6, D5, D4. (Original receiver code used D7..D4 for A/B/C/D control; we preserve that mapping.)

Pin choices avoid any conflict between SPI, I²C, and dimmer.

3) Software (complete, ready to run)

3.1 Libraries to install (Arduino IDE → Library Manager)

RF24 by TMRh20 (for nRF24L01+)

LiquidCrystal_I2C (commonly used with PCF8574 backpacks)

If your LCD address isn't 0x27, change it to 0x3F (or scan for it).

Dimmer control (4-line)

- Dimmer control inputs → **D7, D6, D5, D4** respectively (named A/B/C/D in code).
- Dimmer module power/ground per its patch; load lamp across the dimmer's output as specified by the dimmer board.

Transmitter sketch

```
#include <SPI.h>
#include <RF24.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---- Hardware mapping ----
const int PIN_LDR = A0;
const uint8_t RF_CE_TX  = 7;   // CE
const uint8_t RF_CSN_TX = 8;   // CSN

// I2C LCD: try 0x27 first; change to 0x3F if needed
LiquidCrystal_I2C lcd(0x27, 16, 2);
RF24 radio(RF_CE_TX, RF_CSN_TX);

// 5-byte pipe address (same on RX)
const byte pipeAddr[6] = "L7TX1";

void setup() {
  // LCD
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0,0); lcd.print("LDR Wireless TX");
  lcd.setCursor(0,1); lcd.print("Init...");

  // RF
  radio.begin();
  // Slower rate improves range/robustness in labs
  radio.setDataRate(RF24_250KBPS);
  radio.setRetries(5, 15);    // delay,count
  radio.openWritingPipe(pipeAddr);
  radio.stopListening();       // TX role

  delay(400);
  lcd.setCursor(0,1); lcd.print("Ready           );
}

void loop() {
  int raw = analogRead(PIN_LDR);    // 0..1023
  uint8_t level = raw / 100;        // ~0..10 compact
  // Optional: simple averaging for stability
  // (left out for clarity/latency)

  // Display
  lcd.setCursor(0,0); lcd.print("LDR:      );
  lcd.setCursor(5,0); lcd.print((int)level);
  lcd.setCursor(0,1); lcd.print("Send:      );
  lcd.setCursor(6,1); lcd.print((int)level);

  // Transmit one byte
  radio.write(&level, sizeof(level));

  delay(100);
}
```

Receiver sketch

```
#include <SPI.h>
#include <RF24.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// ---------- Configuration ----------
static const uint8_t RF_CE_PIN  = 2;   // CE
static const uint8_t RF_CSN_PIN = 3;   // CSN
// LED "dimmer" outputs (one-hot)
static const uint8_t LED_A = 7; // Brightest (L4)
static const uint8_t LED_B = 6; // L3
static const uint8_t LED_C = 5; // L2
static const uint8_t LED_D = 4; // Lowest (L1)

// I2C LCD address: try 0x27 first; if blank screen, change to 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

// RF24 radio on CE, CSN
RF24 radio(RF_CE_PIN, RF_CSN_PIN);

// 5-byte (or 6) pipe address — must match transmitter
const byte pipeAddr[6] = "L7TX1";

// ---------- Helper: set exactly one LED ON ----------
static inline void setLevelLEDs(uint8_t levelStep) {
  // levelStep: 1..4 (L1 lowest .. L4 highest)
  bool A=0,B=0,C=0,D=0;
  switch (levelStep) {
    case 1: D = 1; break; // L1 lowest -> LED_D
    case 2: C = 1; break; // L2
    case 3: B = 1; break; // L3
    default: // 4 or anything else -> L4 brightest
      A = 1; break; // LED_A
  }
  digitalWrite(LED_A, A);
  digitalWrite(LED_B, B);
  digitalWrite(LED_C, C);
  digitalWrite(LED_D, D);
}

// ---------- Setup ----------
void setup() {
  // LEDs
  pinMode(LED_A, OUTPUT);
  pinMode(LED_B, OUTPUT);
  pinMode(LED_C, OUTPUT);
  pinMode(LED_D, OUTPUT);
  setLevelLEDs(0); // all off at power-up

  // LCD
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0,0); lcd.print("LDR Wireless RX");
  lcd.setCursor(0,1); lcd.print("Init...");

  // Radio
  radio.begin();
  // Lower data rate improves robustness for student labs
  radio.setDataRate(RF24_250KBPS);
  // Open reading pipe and start listening
  radio.openReadingPipe(1, pipeAddr);
  radio.startListening();

  delay(350);
```

```
  lcd.setCursor(0,1); lcd.print("Listening...    );
}

// ---------- Main Loop ----------
void loop() {
  if (!radio.available()) {
    delay(20);
    return;
  }

  // Drain all pending packets; keep the most recent level
  uint8_t level = 0;
  while (radio.available()) {
    radio.read(&level, sizeof(level));
  }

  // Display the raw level (0..~10 as sent by TX)
  lcd.setCursor(0,0); lcd.print("LDR:      );
  lcd.setCursor(5,0); lcd.print("    );       // clear old digits
  lcd.setCursor(5,0); lcd.print((int)level);

  // Map raw level to 4 discrete steps
  // Tune thresholds as desired for your room/LEDs
  uint8_t step = 4; // default brightest
  if (level <= 2)      step = 1; // L1 (lowest)
  else if (level <= 5) step = 2; // L2
  else if (level <= 8) step = 3; // L3
  else                 step = 4; // L4 (brightest)

  // Update LEDs (one-hot) and LCD label
  setLevelLEDs(step);
  lcd.setCursor(0,1); lcd.print("LED: L");
  lcd.setCursor(6,1); lcd.print(step);
  lcd.setCursor(7,1); lcd.print("         );  // wipe trailing chars

  delay(50);
}
```

Instead of controlling a lamp through a dimmer, the receiver Arduino will **light one of four LEDs** depending on the received light-intensity level.

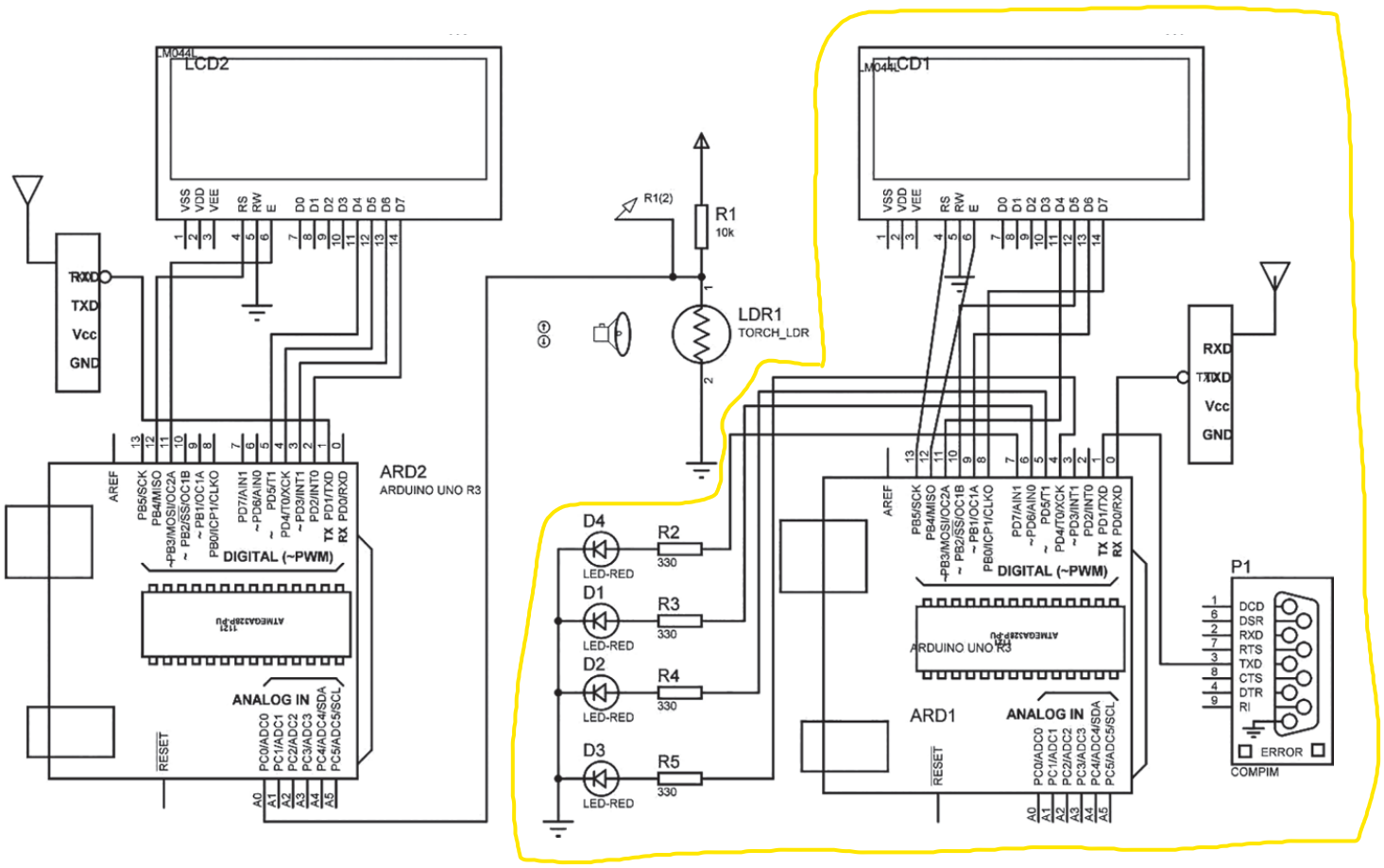
This still represents:
LDR (light level) → Wireless → Control output (brightness state)

Pin Mapping (Receiver)

Arduino Pin	LED	Brightness Level	State in Code
D7	LED A	**Bright**	Level 4
D6	LED B	**Medium-High**	Level 3
D5	LED C	**Medium-Low**	Level 2
D4	LED D	**Low**	Level 1

Only **one** LED turns ON at a time, showing which “brightness step” we are in

Receiver



4) Software

4.1 Transmitter (Arduino Uno)

Why these choices (tied to PDS):

- analogRead(A0) pulls LDR level (0–1023).
- Divide by 100 → compact **0–10**ish byte (fits doc's one-byte send).
- 20×4 LCD shows title + live level for user feedback (PDS: observability).
- Serial.begin(9600) aligns with RF link + LabVIEW settings.

Alternative (optional): If you want PC serial *and* RF simultaneously, swap the RF modem to SoftwareSerial on other pins. The document uses pins 0/1; we keep that to aligned

Step-by-step Procedure (with reasoning)

A) Prep

Install libraries (RF24, LiquidCrystal_I2C).

Assemble power: Provide regulated 5 V for the Uno. Provide 3.3 V to the nRF24 via an adapter/base or LDO + decoupling + level shifters. (Avoid resistor dividers for VCC—voltage sags with burst current.)

Check LCD address: most are 0x27. If you see nothing on the LCD, try 0x3F and adjust contrast pot.

B) Transmitter bring-up

Wire LDR → A0; power the I²C LCD and connect SDA(A4), SCL(A5). (Matches original sensor mapping; LCD is now I²C.)

Wire nRF24 (SPI): CE=7, CSN=8, SCK=13, MOSI=11, MISO=12, VCC=3.3 V.

Upload TX sketch. Cover/uncover the LDR—LCD should show "LDR: x".

C) Receiver bring-up

Wire I²C LCD (A4/A5).

Wire nRF24: CE=2, CSN=3, SCK=13, MOSI=11, MISO=12, VCC=3.3 V.

Wire dimmer inputs to D7..D4 (A/B/C/D). (Same spirit as original.)

Upload RX sketch. When TX is on, RX LCD should show "LDR: x" and "Dim: L1–L4". Change room light over the LDR and observe discrete steps.

D)LabVIEW monitor

Original front-panel concept remains: a serial 9600 8-N-1 viewer is fine for extra credit (you can print debug on RX if you like).

5) Calibration & Extensions

Thresholds: Edit level cut-points (<=2, <=5, <=8) so each step gives a visibly different lamp brightness (LDRs are nonlinear).

16 levels: Map 0–1023 to 0–15, transmit that nibble, and drive the 4 dimmer lines as a binary code (bit-to-pin).

Smoothing: Average several samples on TX if flicker bothers you; tradeoff is added latency.

6) Traceability back to original Lab 7

System goal & blocks: unchanged (LDR → wireless → LCD + dimmer + LabVIEW).

Sensor pin (A0) and basic sampling/divide-by-100 payload: same concept as original Tx code.

Receiver dimmer mapping: same 4-line approach, made exclusive and documented.

Use a **MOSFET-based LED dimmer board** or a **logic-level PWM driver** (0–5 V logic inputs). Each of the four Arduino pins can:

- Directly switch LEDs through small MOSFETs, or
- Act as select lines for a digital potentiometer or DAC that controls LED brightness.

This keeps everything at **5 V** and safe for breadboard demos.

- As the **room light over the LDR** changes, the **receiver's I²C LCD** updates the "Dim: L1–L4" line.
- At the same time, the **lamp/LED** connected to the dimmer will brighten or dim in discrete steps.
- This demonstrates the full closed-loop behavior:
  LDR → Transmitter → Wireless → Receiver → Dimmer → Load.

A) Transmitter bring-up

1. **Power check**: verify +5 V and GND rails before inserting modules — reduces risk to LCD/RF. (Meets PDS reliability.)
2. **Wire LDR to A0** as specified above. (Aligns with doc's analog input choice.)
3. **Wire LCD** per the transmitter constructor mapping. Students must learn that **code and wiring must match** the LiquidCrystal(...) pin order. (Traceability to PDS observability.)
4. **Temporarily disconnect RF TX/RX** from pins 0/1 and **upload Tx sketch**. Reconnect after upload. (Avoid serial contention.)
5. **Sanity test**: Cover/uncover LDR and confirm "LDR LEVEL" changes on LCD. The range should move ~0–10. If stuck, check LDR wiring and LCD RS/E lines.

B) Receiver bring-up

1. **Wire LCD** per receiver constructor mapping (different from Tx per the doc).
2. **Wire dimmer** inputs to D7..D4. Keep the **load isolated** and supervised. (Meets PDS safety.)
3. **RF modem on 0/1**, but **disconnect during upload** of the Rx sketch.
4. **Power receiver only** first; LCD should show "Receiver".
5. **Then power transmitter**; verify Rx LCD prints incoming "LDR LEVEL".
6. **Observe dimmer steps** while changing light over the LDR. Explain to students how thresholds map to output codes.

C) Wireless link verification

- If LCD values update but dimmer doesn't: check D4–D7 wiring vs code.
- If no LCD update: check **baud 9600** on both sides and RF power/GND. (Doc sets 9600 8N1 across the system.)
- Port: COMx talking to the receiver (or a sniffer), **9600 baud**, **8 data bits**, **no parity**, **1 stop bit**, **no flow control**. Use a Serial Read block, an indicator/string/Chart to show the byte stream (count ~100), and optionally a waveform chart.

Tip: If the RF modem occupies pins 0/1 on the Arduino, you'll view the stream from the PC only if you route a second serial path or temporarily plug the Rx Arduino directly to PC and forward bytes. The document's intent is a simple front panel aligned to 9600-8-N-1.

7) Testing & calibration guidance

- **Threshold tuning:** In the Rx sketch, adjust bins (e.g., <=2, <=5, <=8) to give approximately equal visual jumps in lamp brightness. Rationale: the LDR response is nonlinear; bins guarantee repeatable demos.
- **Noise/averaging (optional):** At Tx, average several analogRead() samples to smooth flicker before dividing by 100 (trade latency vs stability).
- **Range scaling (optional):** Instead of /100, map 0–1023 → 0–15 and drive **4-bit dimmer code** directly for 16 levels.

5 LabVIEW GUI

Before Starting Labview Please watch the following videos to understand how to communicate Arduino with labview!

https://www.youtube.com/watch?v=ODJOwD8WklI&list=PLPxqQA3MmpV6wpObVs3D24XdbYmu1FsDL

https://www.youtube.com/watch?v=FNUG8n0IUz8&list=PLPxqQA3MmpV6wpObVs3D24XdbYmu1FsDL&index=2

https://www.youtube.com/watch?v=qQqPCWeB_QY&list=PLPxqQA3MmpV6wpObVs3D24XdbYmu1FsDL&index=3

LabVIEW GUI is designed for LDR-based light-intensity control system. GUI is designed in two parts: (1) block diagram and the (2) front panel. The front panel has a serial COM port to access the data from the PC port, baud rate is set as "9600," which is same as the wireless modem "Zigbee" to receive the data with the same rate, data bits "8," parity "none" stop bits "1," serial count is set as "100," flow control "None," and Serial_read_buffer to read the received data from the fire sensor. Waveform chart is optional; it can be used to check the data received in the form of waveform. To make the front panel, an active graphical programming is done in the block diagram (Figures 4 through 5)

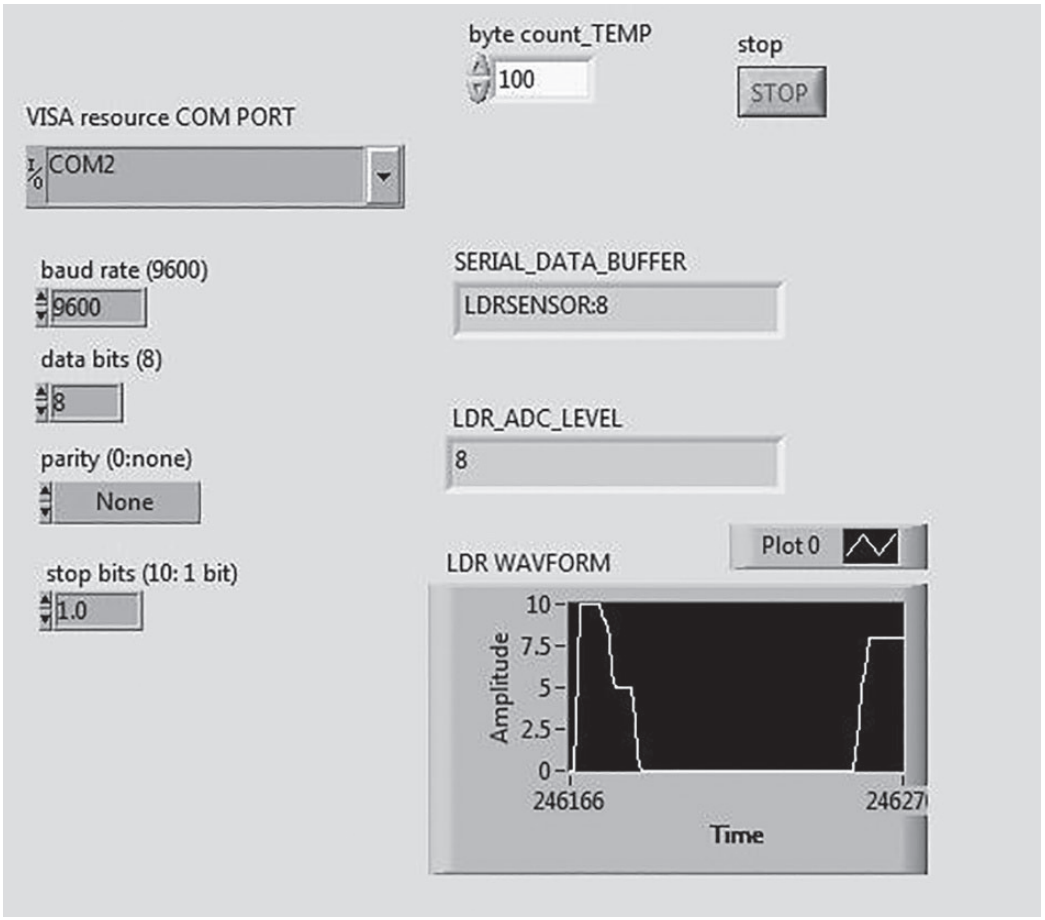


FIGURE 4
Front panel for the system.

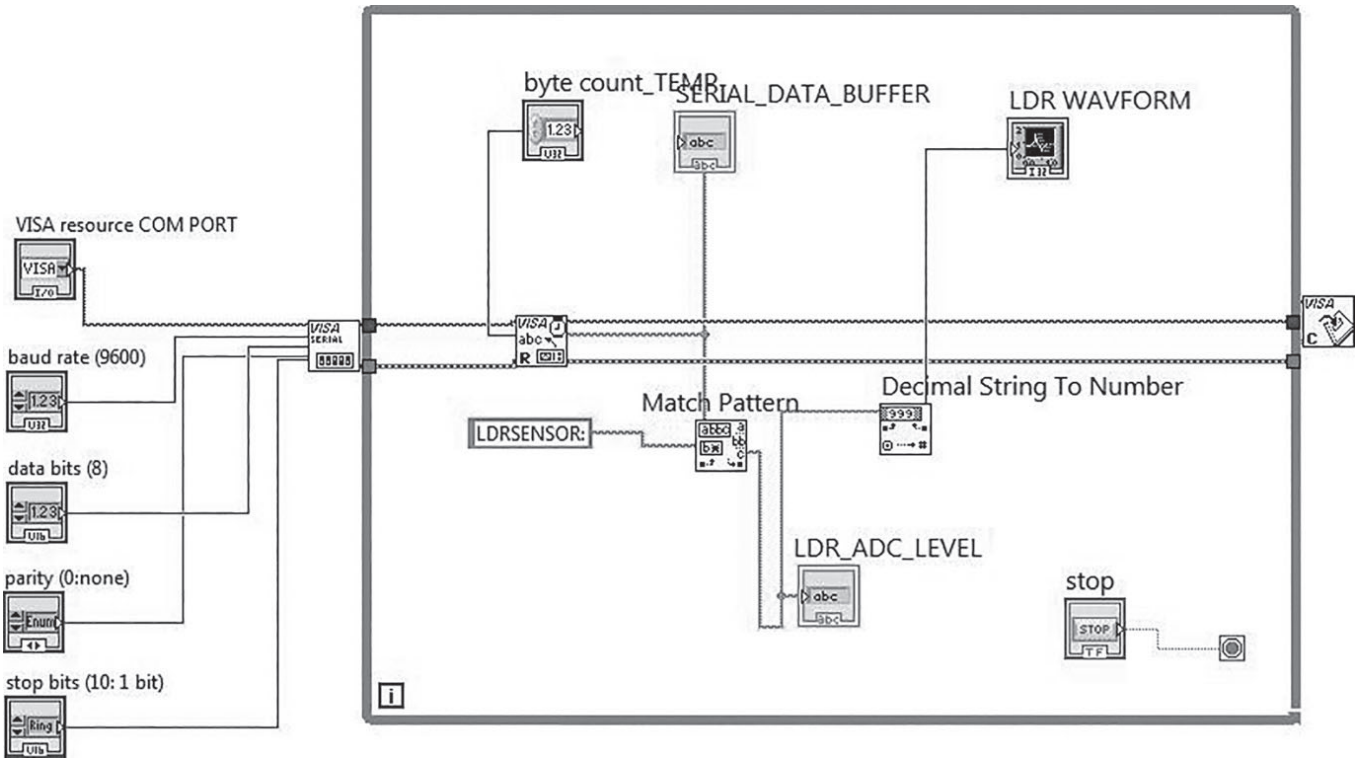


FIGURE 5
LabVIEW GUI block diagram for the system.